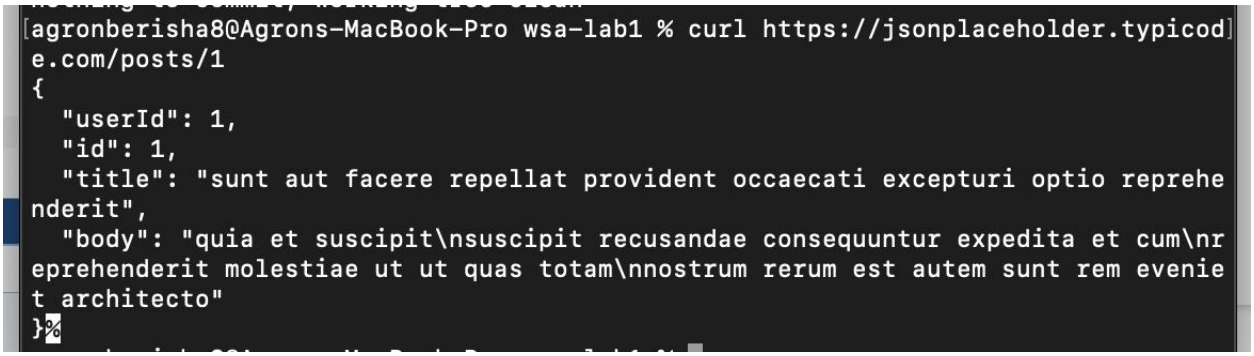
Lab 1

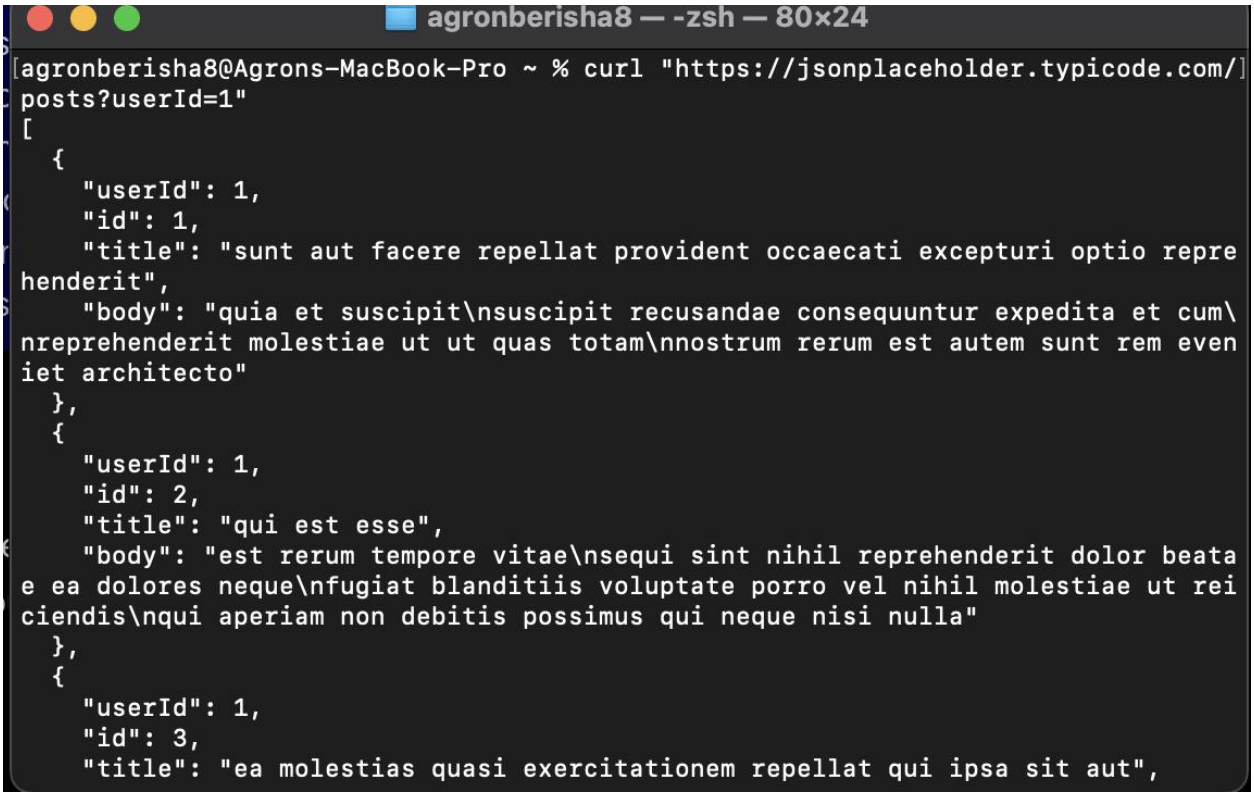
A1 – It comes on JSON format, the data that comes are from the post 1, the userId, id, title, body



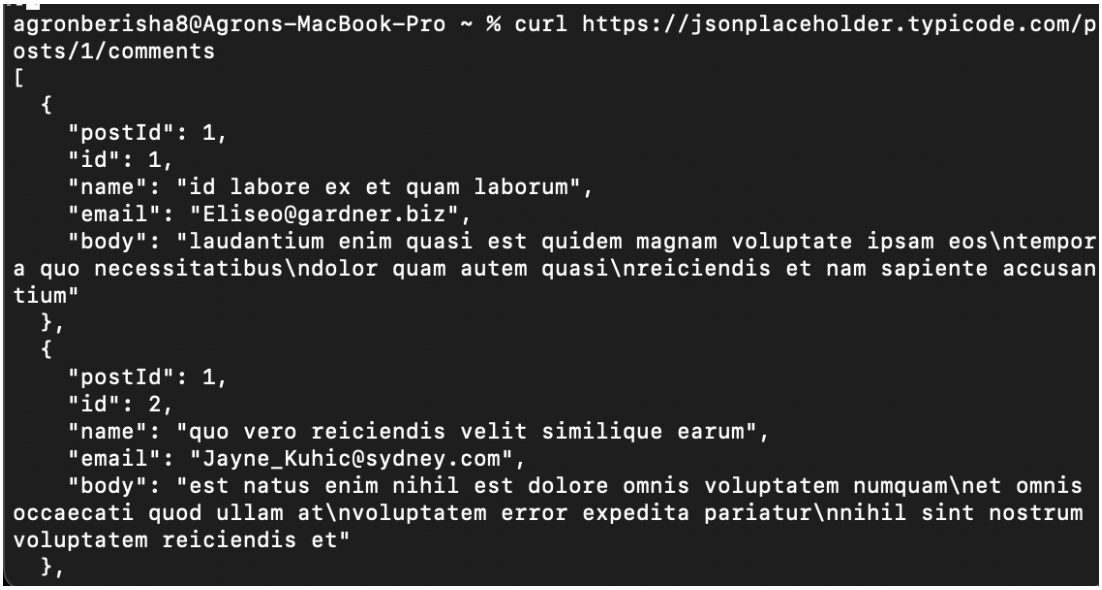
A2 -

```
agronberisha8@Agrons-MacBook-Pro wsa-lab1 % curl -i https://jsonplaceholder.typicode.com/posts/1
HTTP/2 200
date: Mon, 12 Oct 2026 15:57:58 GMT
content-type: application/json; charset=utf-8
content-length: 292
access-control-allow-credentials: true
cache-control: max-age=43200
etag: W/"124-yiKdLzqO5gfBrJFrcdJ8Yq0LGnU"
expires: -1
pragma: no-cache
vary: Origin, Accept-Encoding
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 3600000
x-ratelimit-remaining: 3599999
x-ratelimit-reset: 1790812826
age: 21731
accept-ranges: bytes
cf-cache-status: HIT
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=rsX4pwjtYe0tMMRLJqdqoRPojoAJQGmW8f7wzkcH2W0DJ4Ac2eeQAefuVY%2Fd76Nv7aYp9B8dcJy33gH2qfPsdr6JKkbMValNkDZ826M%2FCg83vhqZFam4qVTvaL0St02LxZw9CIG7QbPh6w5nAZPi"}]}
```

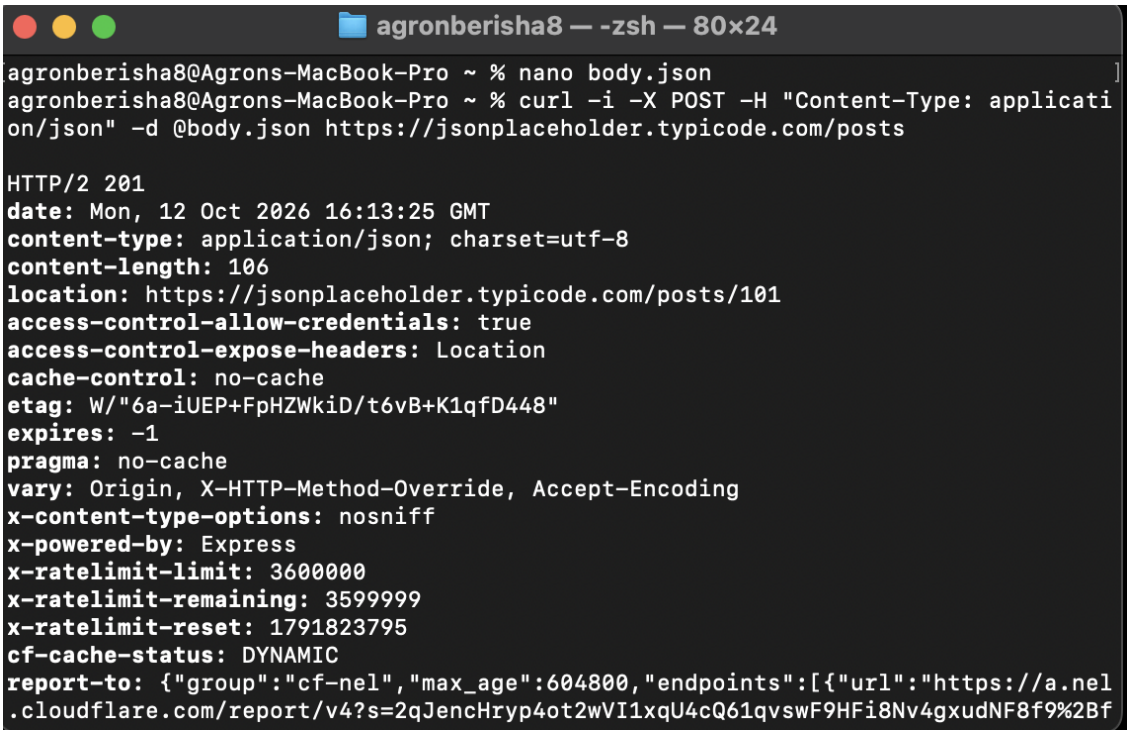
A3 – it shows 10 posts from the userId: 1, every post that userId 1 makes it brings back



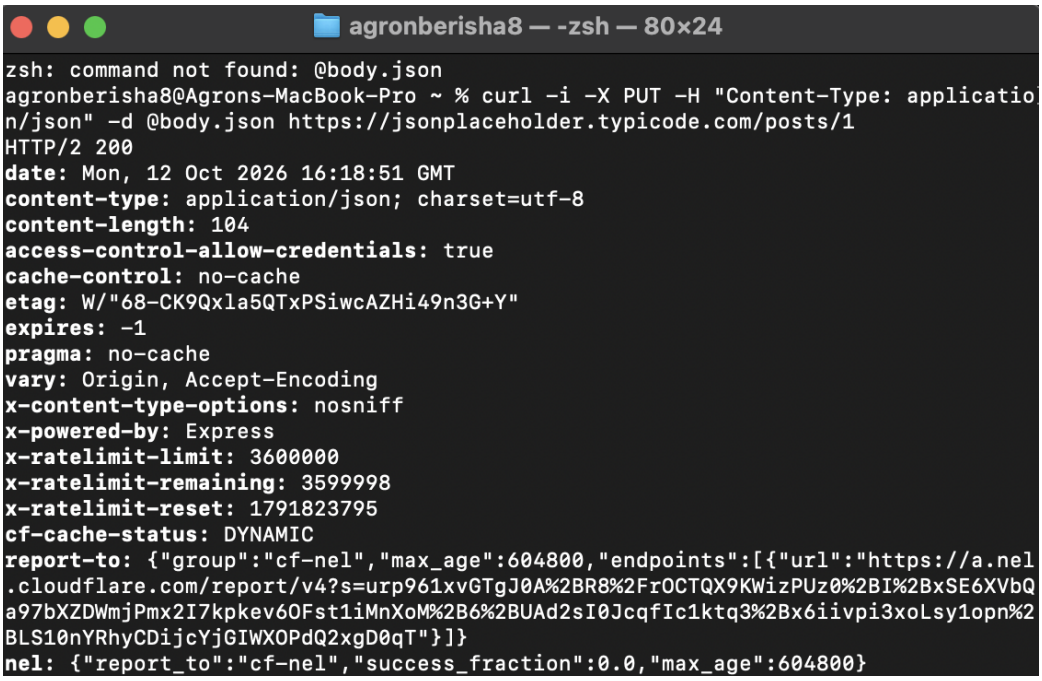
A4- this API gets the data that comments that are related with the posts user 1 that has been made and it fetches.



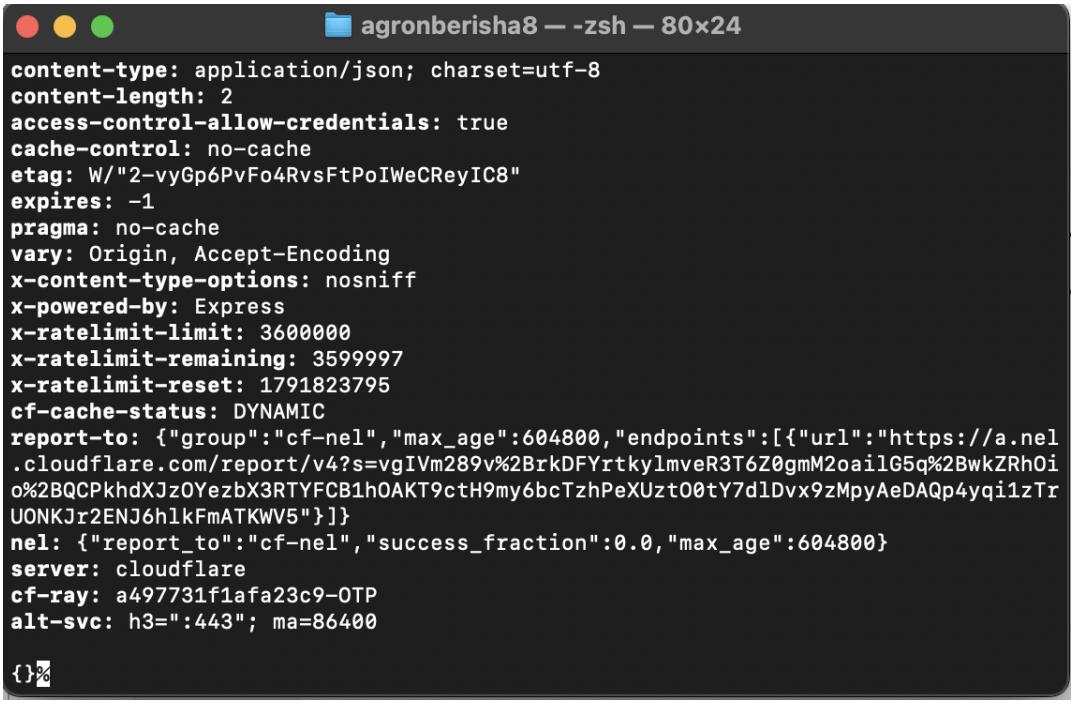
A5- we just did send API post to jsonplaceholder, we created first the file, and then send post json



A6- we updated the post with the method PUT



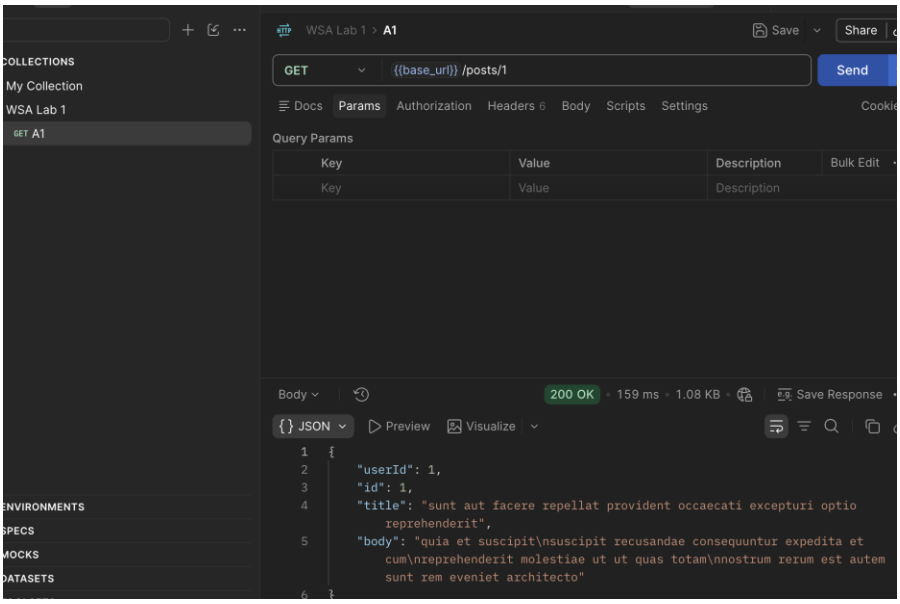
A7- We just deleted the post id 1, it returns nothing not body not nothing data



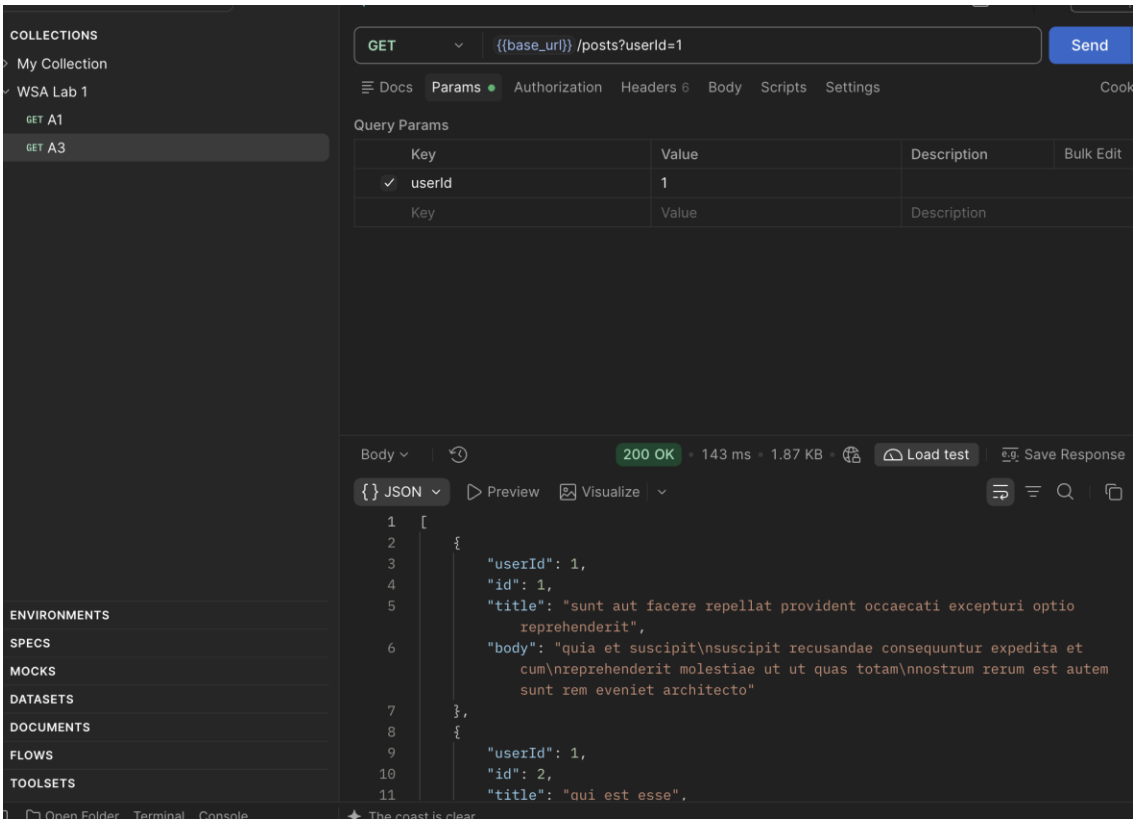
A8- it returns Error 404, that it does not exist, and it shows nothing inside the body

```
agronberisha8 — -zsh — 80×24
agronberisha8@Agrons-MacBook-Pro ~ % curl -i https://jsonplaceholder.typicode.com/posts/9999
HTTP/2 404
date: Mon, 12 Oct 2026 16:25:26 GMT
content-type: application/json; charset=utf-8
content-length: 2
access-control-allow-credentials: true
cache-control: max-age=43200
etag: W/"2-vyGp6PvFo4RvsFtPoIWeCReyIC8"
expires: -1
pragma: no-cache
vary: Origin, Accept-Encoding
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 3600000
x-ratelimit-remaining: 3599991
x-ratelimit-reset: 1791803935
nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
cf-cache-status: HIT
report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=qBSc%2BZLKWTWeMbRy1nUM5pPNVcD8QBKA0PSrmkekP3GiohAjHETT2V7Xsdp7cYbL0Ly53ZMbk6KPf%2F%2F20HwVmloSXkDoJCwswwpGfyXmgIYuu7%2FtWcL0KpEuUYs9HZHfGpkGdVBs66KBylELHdiN"}]}
server: cloudflare
```

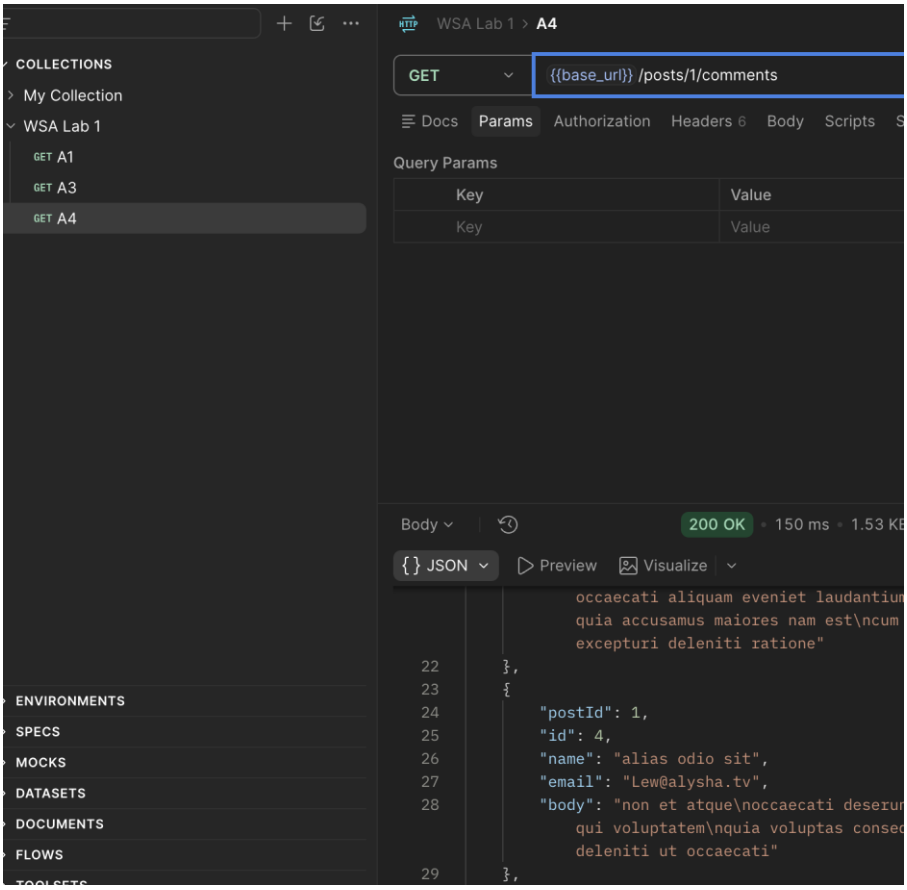
B1:



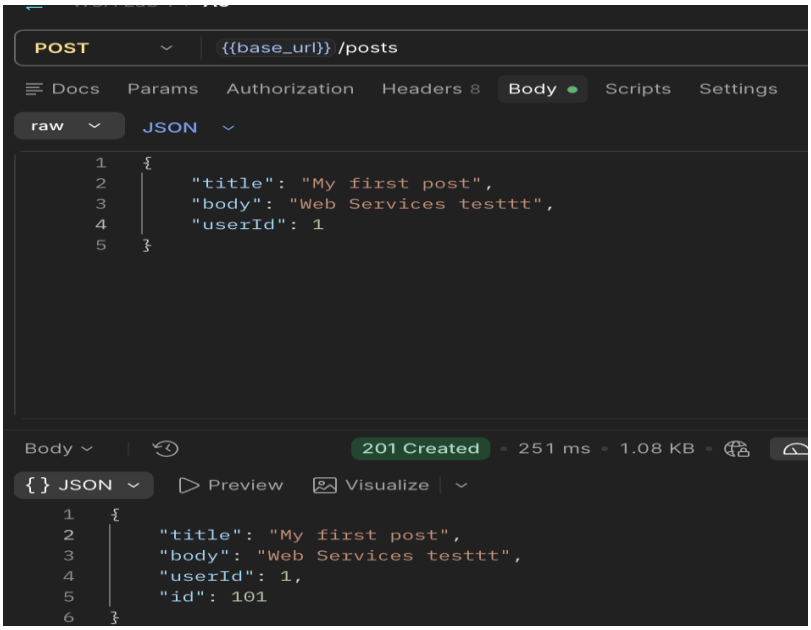
B2:


COLLECTIONS
My Collection
WSA Lab 1
GET A1
GET A3
ENVIRONMENTS
SPECS
MOCKS
DATASETS
DOCUMENTS
FLOWS
TOOLSETS
GET
{{base_url}} /posts?userId=1
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Query Params
Key
Value
Description
Bulk Edit
userId
1
Key
Value
Description
Body
200 OK
143 ms
1.87 KB
Load test
Save Response
{ } JSON
Preview
Visualize
1 [
2 {
3 "userId": 1,
4 "id": 1,
5 "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6 "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7 },
8 {
9 "userId": 1,
10 "id": 2,
11 "title": "qui est esse",

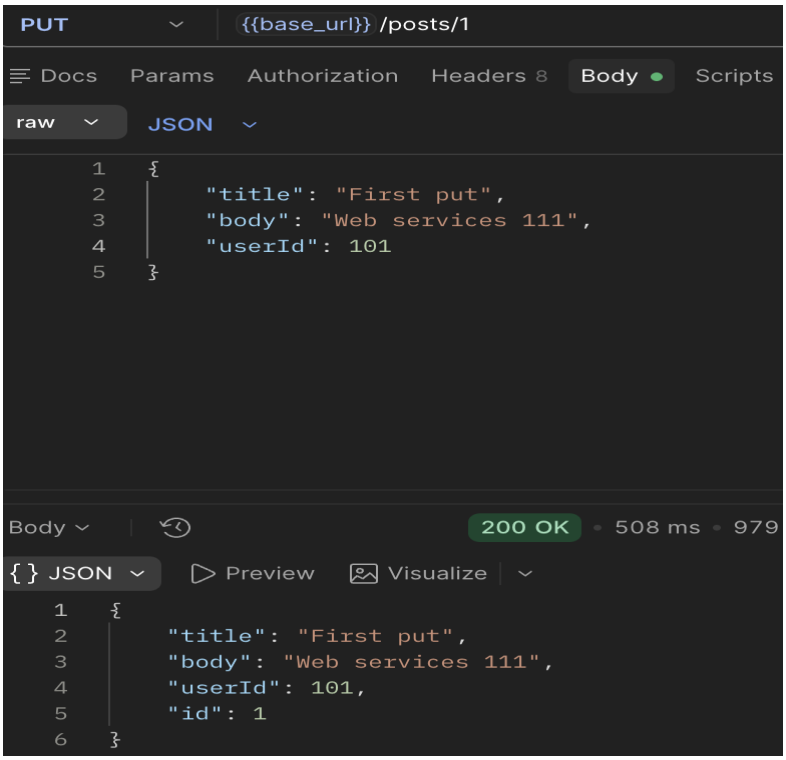
B3:


COLLECTIONS
My Collection
WSA Lab 1
GET A1
GET A3
GET A4
ENVIRONMENTS
SPECS
MOCKS
DATASETS
DOCUMENTS
FLOWS
WSA Lab 1 › A4
GET
{{base_url}} /posts/1/comments
Docs
Params
Authorization
Headers 6
Body
Scripts
Query Params
Key
Value
Key
Value
Body
200 OK
150 ms
1.53 KB
{ } JSON
Preview
Visualize
occaecati aliquam eveniet laudantium
quia accusamus maiores nam est\ncum
excepturi deleniti ratione"
},
{
"postId": 1,
"id": 4,
"name": "alias odio sit",
"email": "Lew@alysha.tv",
"body": "non et atque\noccaecati deserun
qui voluptatem\nquia voluptas consec
deleniti ut occaecati"
},

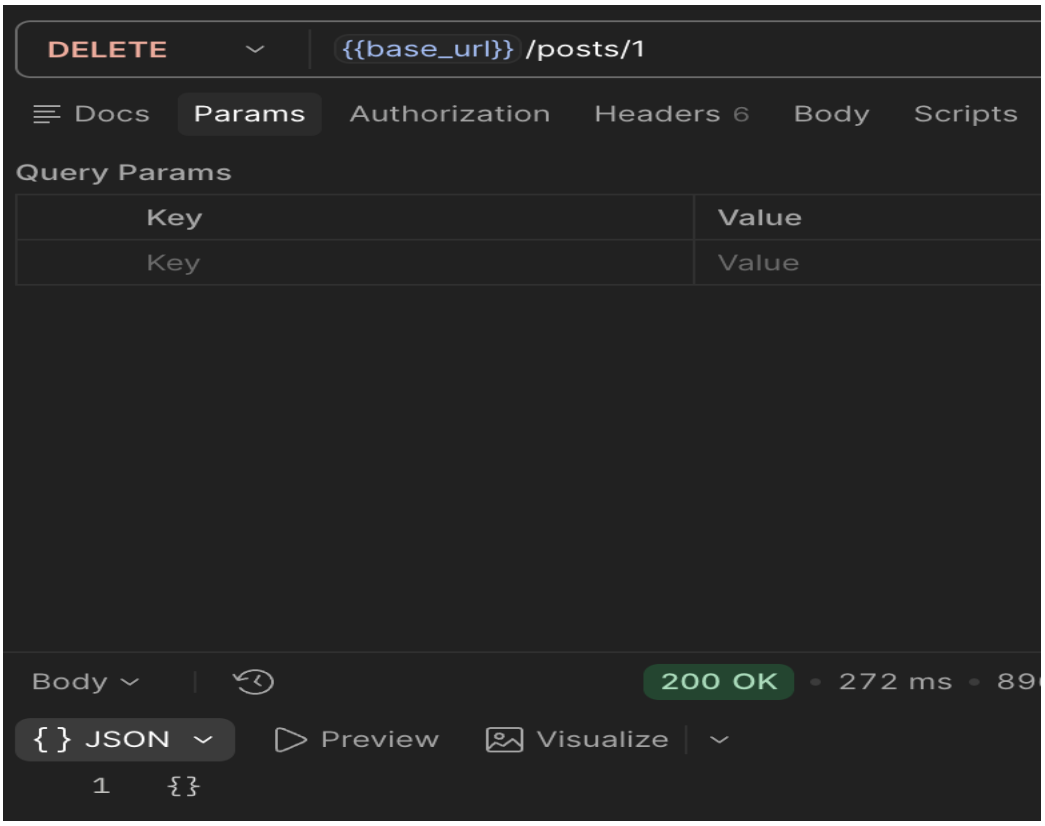
B4:


POST
{{base_url}} /posts
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
raw
JSON
{
"title": "My first post",
"body": "Web Services testtt",
"userId": 1
}
Body
201 Created
251 ms
1.08 KB
{ } JSON
Preview
Visualize
{
"title": "My first post",
"body": "Web Services testtt",
"userId": 1,
"id": 101
}

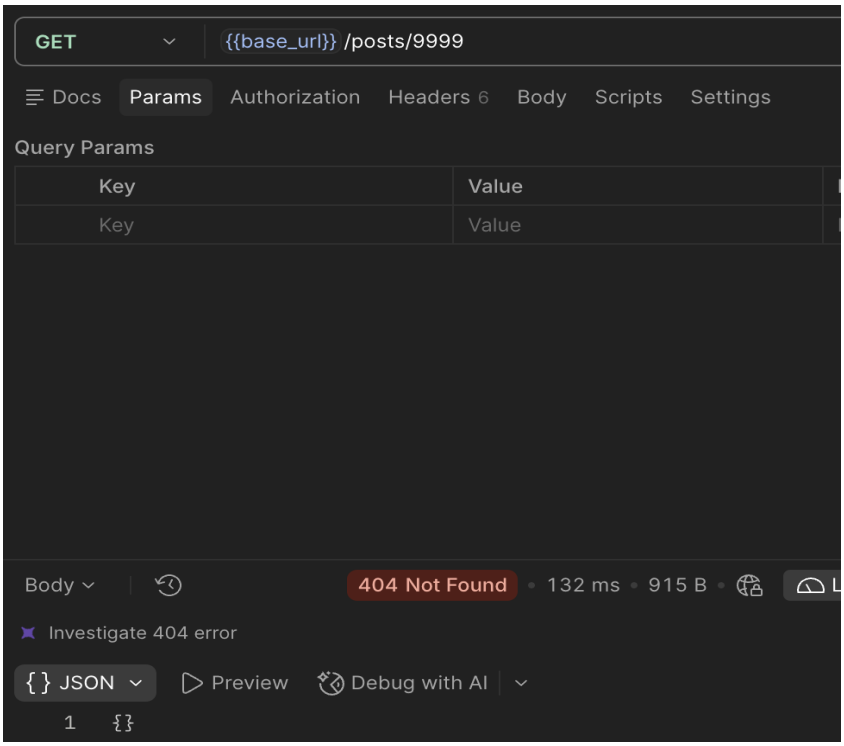
B5:


PUT
{{base_url}} /posts/1
Docs
Params
Authorization
Headers 8
Body
Scripts
raw
JSON
{
"title": "First put",
"body": "Web services 111",
"userId": 101
}
Body
200 OK
508 ms
979
{ } JSON
Preview
Visualize
{
"title": "First put",
"body": "Web services 111",
"userId": 101,
"id": 1
}

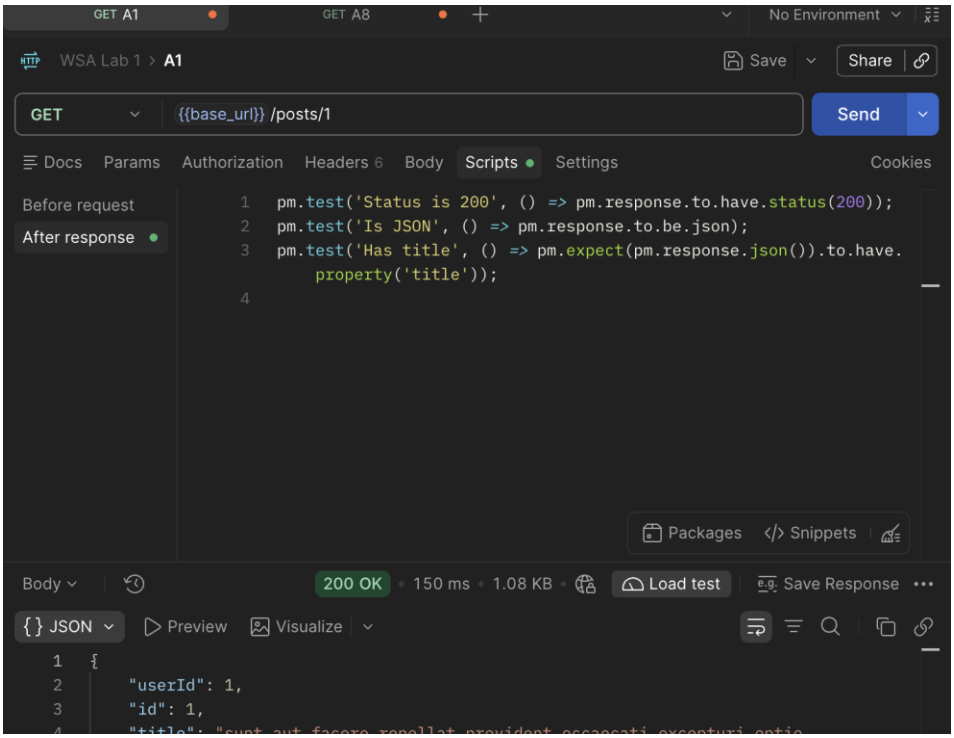
B6:


DELETE
{{base_url}} /posts/1
Docs
Params
Authorization
Headers 6
Body
Scripts
Query Params
Key
Value
Key
Value
Body
200 OK
272 ms
{ } JSON
Preview
Visualize
1 {}

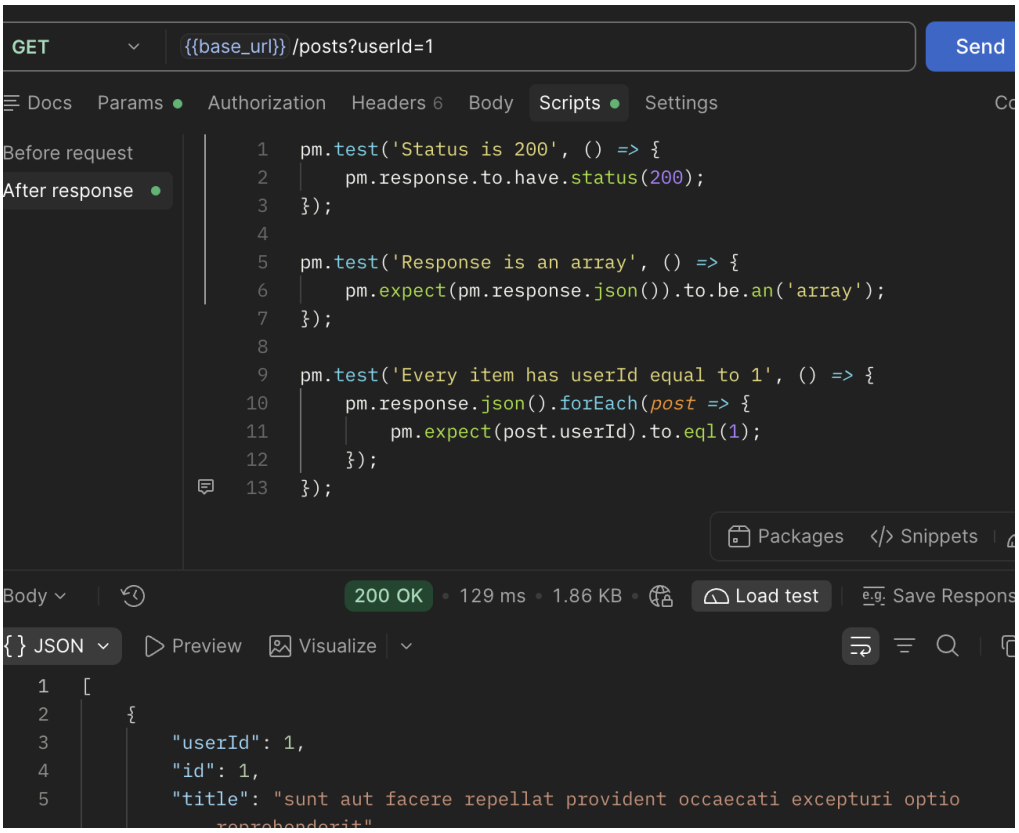
B7:


GET
{{base_url}} /posts/9999
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Query Params
Key
Value
Key
Value
Body
404 Not Found
132 ms
915 B
Investigate 404 error
{ } JSON
Preview
Debug with AI
1 {}

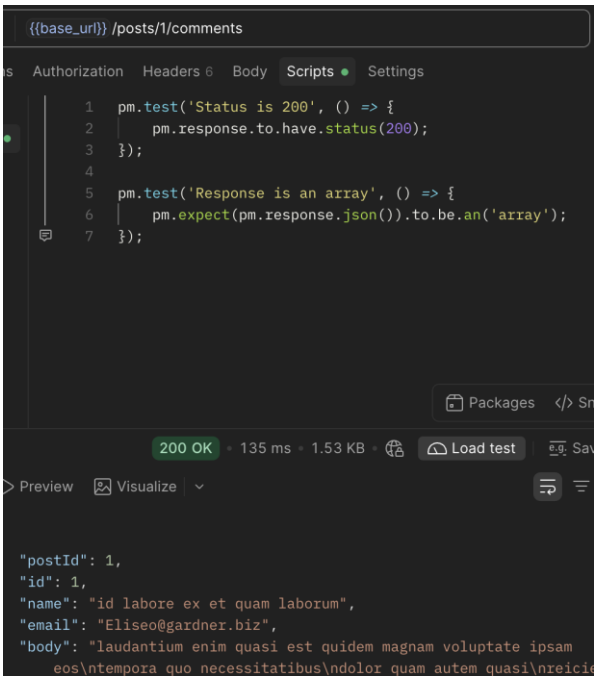
D1:


GET A1
GET A8
No Environment
WSA Lab 1 > A1
Save
Share
GET
{{base_url}} /posts/1
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Cookies
Before request
After response
1 pm.test('Status is 200', () => pm.response.to.have.status(200));
2 pm.test('Is JSON', () => pm.response.to.be.json);
3 pm.test('Has title', () => pm.expect(pm.response.json()).to.have.
property('title'));
4
Packages
Snippets
Body
200 OK
150 ms
1.08 KB
Load test
Save Response
{ } JSON
Preview
Visualize
1 {
2 "userId": 1,
3 "id": 1,

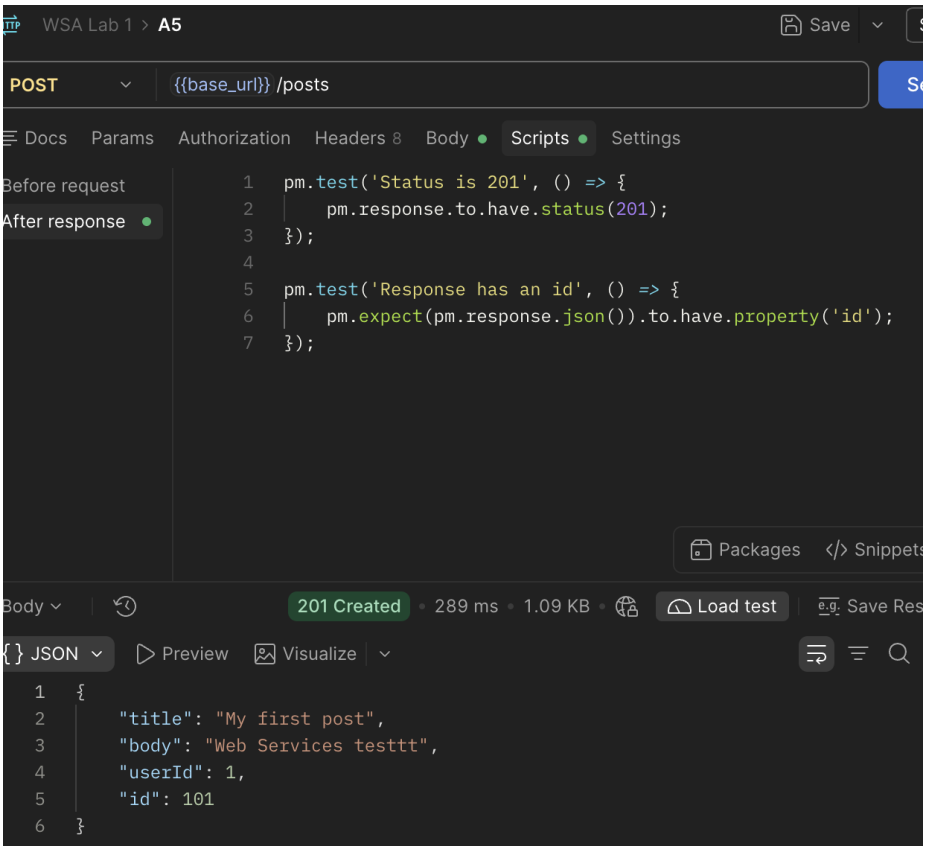
D2:


GET
{{base_url}} /posts?userId=1
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Before request
After response
pm.test('Status is 200', () => {
pm.response.to.have.status(200);
});
pm.test('Response is an array', () => {
pm.expect(pm.response.json()).to.be.an('array');
});
pm.test('Every item has userId equal to 1', () => {
pm.response.json().forEach(post => {
pm.expect(post.userId).to.eql(1);
});
});
Packages
Snippets
Body
200 OK
129 ms
1.86 KB
Load test
Save Respons
JSON
Preview
Visualize
[
{
"userId": 1,
"id": 1,
"title": "sunt aut facere repellat provident occaecati excepturi optio

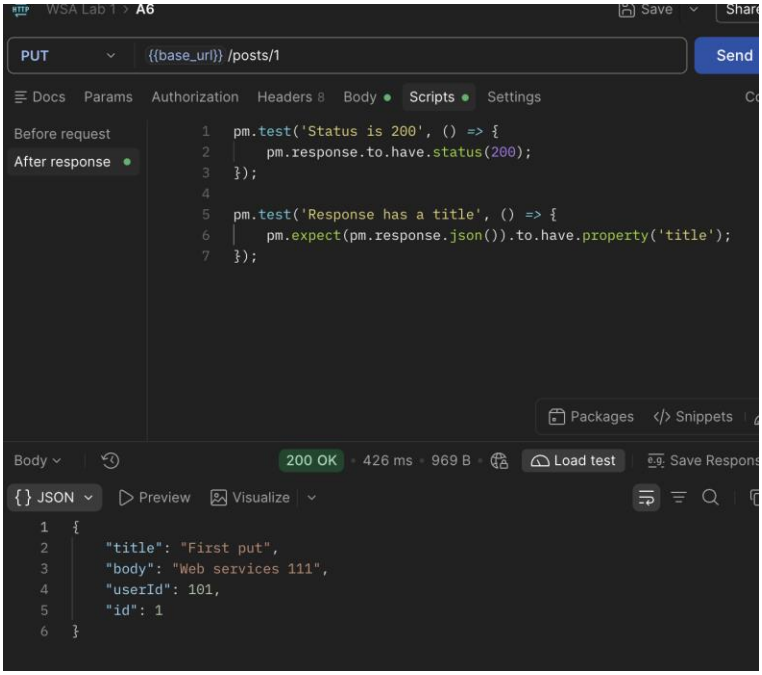
D3:


{{base_url}} /posts/1/comments
Authorization
Headers 6
Body
Scripts
Settings
pm.test('Status is 200', () => {
pm.response.to.have.status(200);
});
pm.test('Response is an array', () => {
pm.expect(pm.response.json()).to.be.an('array');
});
Packages
200 OK
135 ms
1.53 KB
Load test
Preview
Visualize
"postId": 1,
"id": 1,
"name": "id labore ex et quam laborum",
"email": "Eliseo@gardner.biz",
"body": "laudantium enim quasi est quidem magnam voluptate ipsam
eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreici

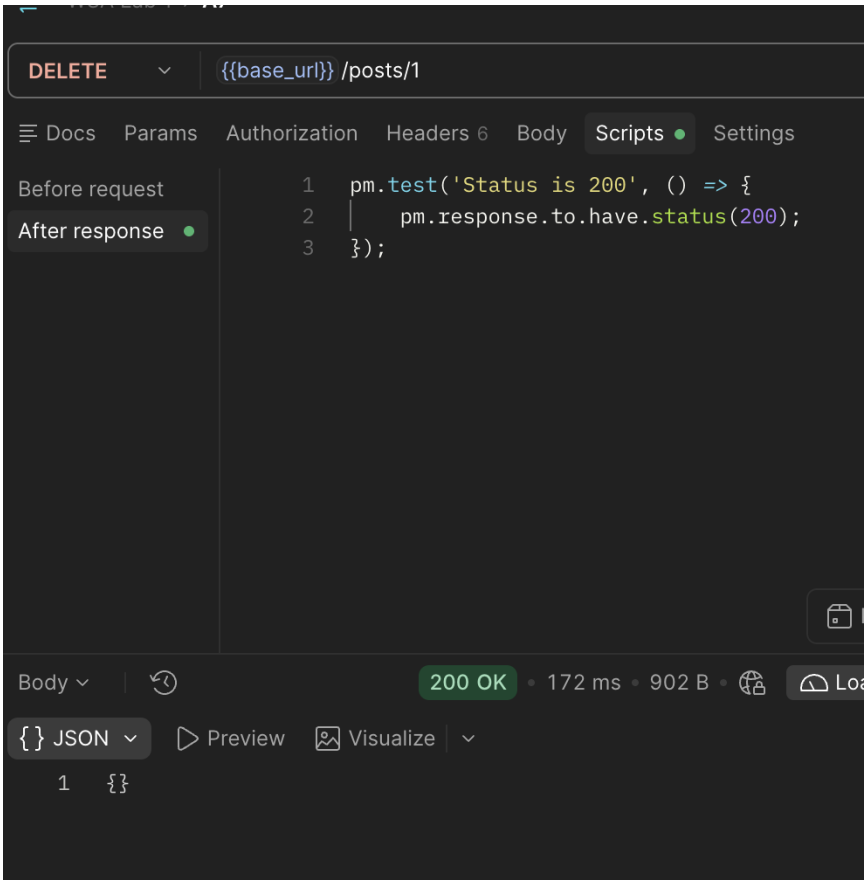
D4:


WSA Lab 1 > A5
Save
POST
{{base_url}} /posts
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Before request
After response
pm.test('Status is 201', () => {
pm.response.to.have.status(201);
});
pm.test('Response has an id', () => {
pm.expect(pm.response.json()).to.have.property('id');
});
Packages
Snippets
Body
201 Created
289 ms
1.09 KB
Load test
Save Res
{ } JSON
Preview
Visualize
{
"title": "My first post",
"body": "Web Services testtt",
"userId": 1,
"id": 101
}

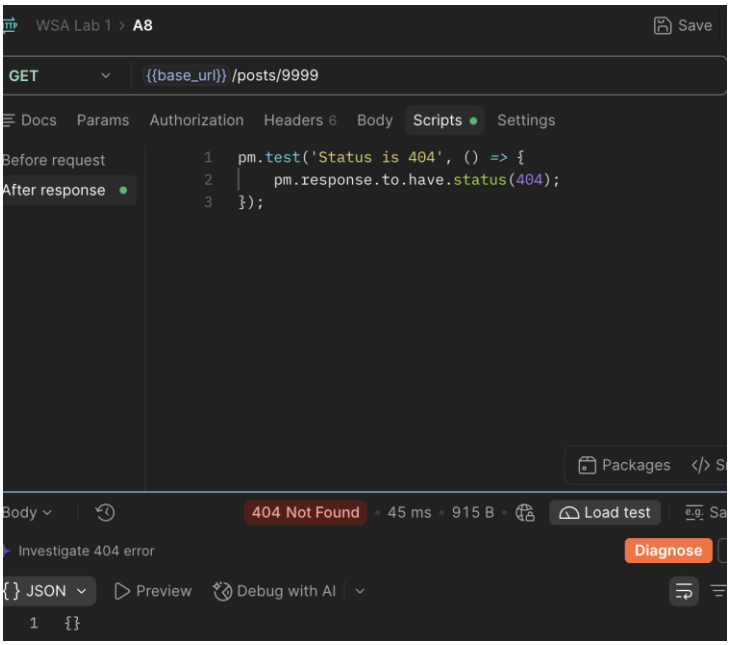
D5:


WSA Lab 1 > A6
Save
PUT
{{base_url}} /posts/1
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Before request
After response
pm.test('Status is 200', () => {
pm.response.to.have.status(200);
});
pm.test('Response has a title', () => {
pm.expect(pm.response.json()).to.have.property('title');
});
Packages
Snippets
Body
200 OK
426 ms
969 B
Load test
{ } JSON
Preview
Visualize
{
"title": "First put",
"body": "Web services 111",
"userId": 101,
"id": 1
}

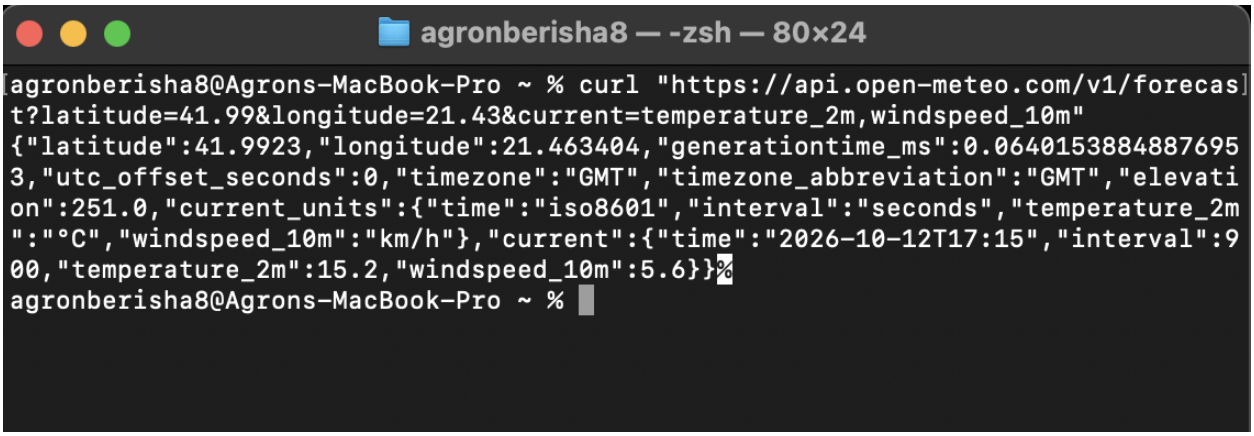
D6:

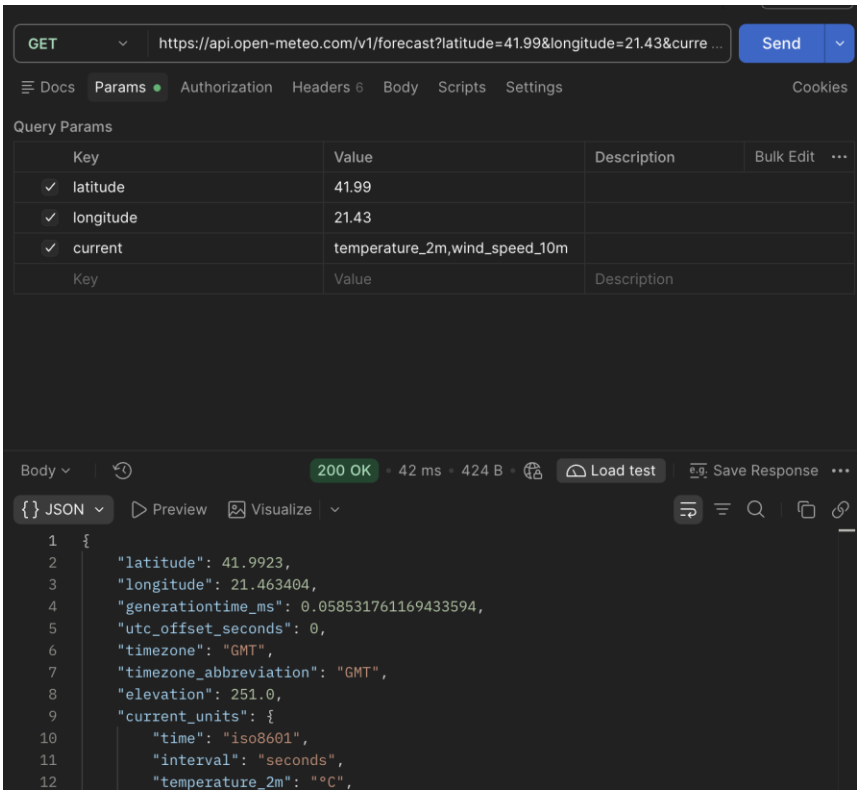

DELETE
{{base_url}} /posts/1
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Before request
After response
1 pm.test('Status is 200', () => {
2 pm.response.to.have.status(200);
3 });
Body
200 OK
172 ms
902 B
{} JSON
Preview
Visualize
1 {}

D7:

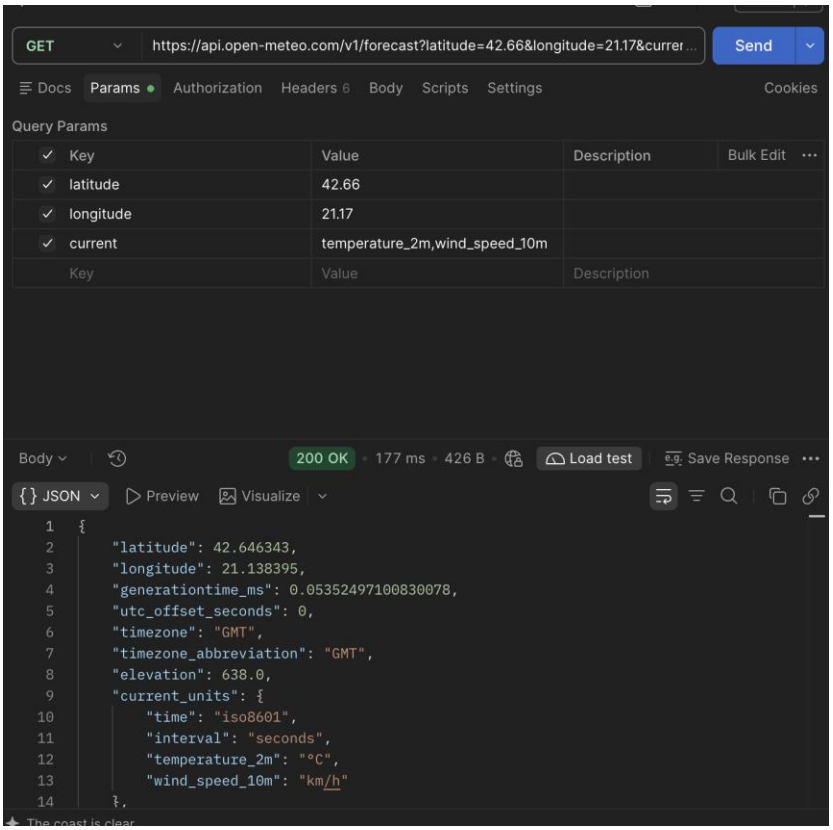

WSA Lab 1 > A8
Save
GET
{{base_url}} /posts/9999
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Before request
After response
1 pm.test('Status is 404', () => {
2 pm.response.to.have.status(404);
3 });
Packages
Body
404 Not Found
45 ms
915 B
Load test
Investigate 404 error
Diagnose
{} JSON
Preview
Debug with AI
1 {}

D1:

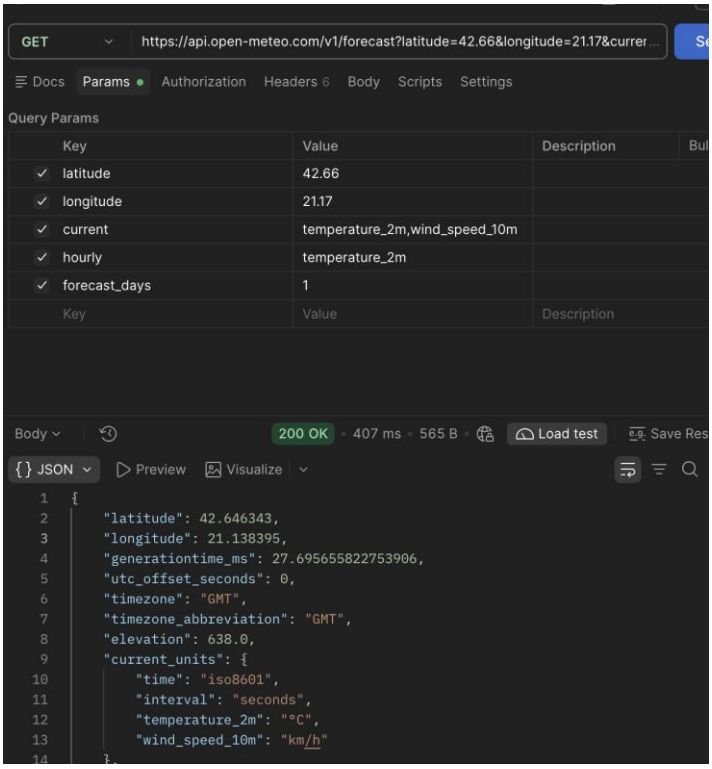

agronberisha8 — -zsh — 80×24
agronberisha8@Agrons-MacBook-Pro ~ % curl "https://api.open-meteo.com/v1/forecast?latitude=41.99&longitude=21.43&current=temperature_2m,windspeed_10m"
{"latitude":41.9923,"longitude":21.463404,"generationtime_ms":0.06401538848876953,"utc_offset_seconds":0,"timezone":"GMT","timezone_abbreviation":"GMT","elevation":251.0,"current_units":{"time":"iso8601","interval":"seconds","temperature_2m":"°C","windspeed_10m":"km/h"},"current":{"time":"2026-10-12T17:15","interval":900,"temperature_2m":15.2,"windspeed_10m":5.6}}%
agronberisha8@Agrons-MacBook-Pro ~ %


GET
https://api.open-meteo.com/v1/forecast?latitude=41.99&longitude=21.43&curre ...
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
latitude
41.99
longitude
21.43
current
temperature_2m,wind_speed_10m
Key
Value
Description
Body
200 OK
42 ms
424 B
Load test
Save Response
{} JSON
Preview
Visualize
{
"latitude": 41.9923,
"longitude": 21.463404,
"generationtime_ms": 0.05853176116943359,
"utc_offset_seconds": 0,
"timezone": "GMT",
"timezone_abbreviation": "GMT",
"elevation": 251.0,
"current_units": {
"time": "iso8601",
"interval": "seconds",
"temperature_2m": "°C",

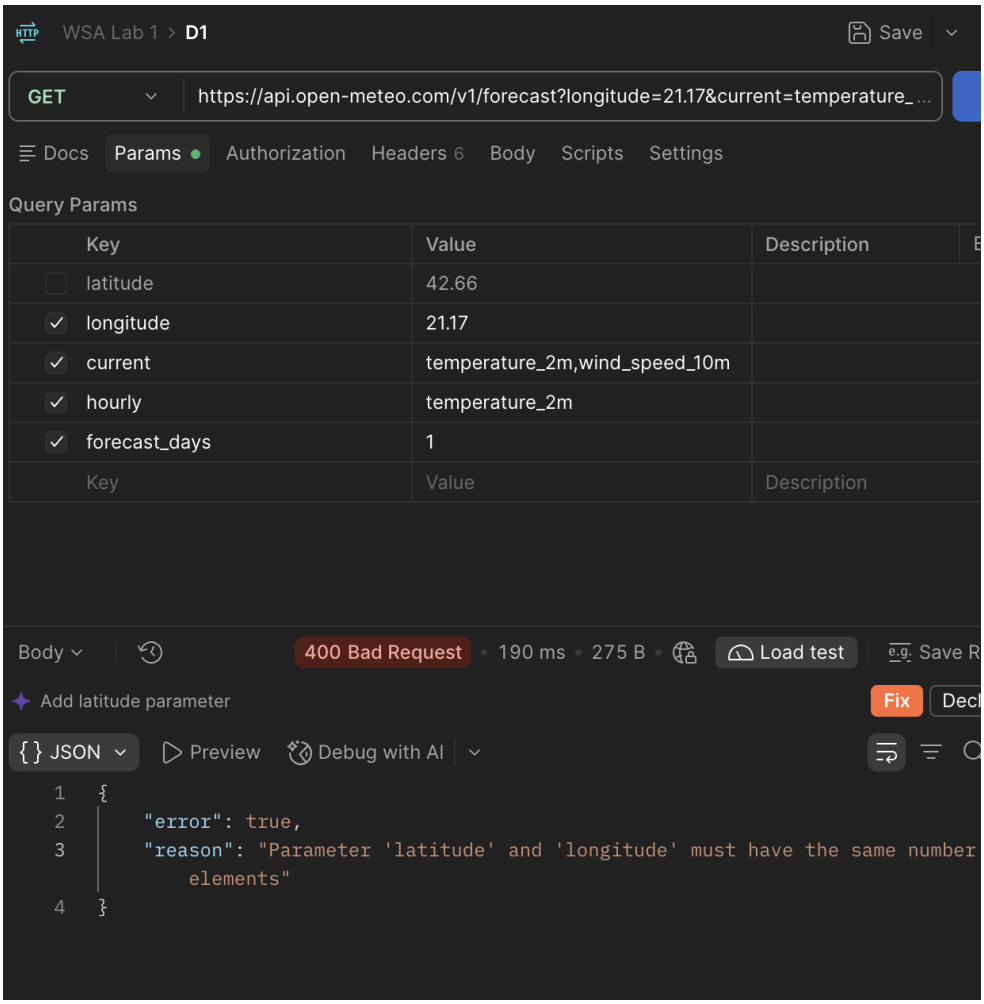
D2: now i changed the values according to the Pristina values longitude and latitude



D3: Added now two new parameters: forecast and hourly



D4: I removed unchecked the labeled longtitude and it failed 400



E1: The status line tells us if the request was successful, for example, 200 means OK and 404 means not found. The headers give extra information, like the content type and the format of the response, before we read the body.

E2: A path parameter identifies a specific resource, like /posts/1, which gets post number 1. A query parameter filters the results, like /posts?userId=1, which gets posts belonging to user 1. I would use a path parameter to find one specific item and a query parameter to filter or search data.

E3: JSONPlaceholder is a test API, so it simulates creating a post but does not permanently save it. To check if a real API stored the data, I would send a POST request and then send a GET request to check if the new post exists.

E4: Open-Meteo returns JSON, while the SOAP GetWeather example uses XML and a SOAP message structure. Open-Meteo is simpler to read and use, while SOAP supports formal contracts through WSDL and has standard features for things like security and reliable messaging.